



# RAPPORT FINALE

## Générateur de grilles de Sudoku à difficulté contrôlée

### Groupe X

KONCHEV Martin

DOLO Abdourahamane-Abdoul

BARRE-GUEROT Ilan

BRIOT-AMADIUE Kellyan

**Encadrant :** Marc Hartley

*L3 informatique*

Faculté des Sciences

Université de Montpellier

Avril 2026

# 1 Présentation du Sujet

## 1.1 Contexte et problématique

Le projet s'inscrit dans le domaine de la conception d'outils algorithmiques appliqués aux jeux logiques, et plus particulièrement au Sudoku. Ce jeu, largement répandu, consiste à remplir une grille  $9 \times 9$  en respectant des contraintes strictes : chaque chiffre doit apparaître une seule fois par ligne, par colonne et par sous-grille  $3 \times 3$ .

La problématique principale étudiée dans ce projet est la **génération automatique de grilles de Sudoku avec une difficulté contrôlée**, tout en garantissant qu'elles possèdent une solution unique. Ce problème est non trivial, car il implique plusieurs défis combinés : générer des grilles valides, contrôler leur complexité, et être capable de les résoudre efficacement pour en analyser la difficulté.

Dans le contexte du projet, l'objectif est de concevoir un système complet intégrant :

- un générateur de grilles,
- un solveur automatique,
- un mécanisme d'évaluation de la difficulté,
- ainsi qu'une interface permettant de manipuler les grilles.

Comme mentionné dans le rapport de Sprint 1, le projet vise à créer un outil capable de produire, analyser et manipuler des grilles de Sudoku de manière automatisée.

## 1.2 Intérêt du sujet

Ce projet présente un intérêt à la fois pédagogique et technique.

D'un point de vue académique, il mobilise plusieurs notions fondamentales de l'informatique :

- les algorithmes de recherche (notamment le backtracking),
- la gestion de structures de données (matrices représentant les grilles),
- l'analyse de complexité,
- et la conception logicielle modulaire.

Il s'inscrit pleinement dans le parcours de Licence Informatique, en permettant d'appliquer concrètement des concepts théoriques étudiés en cours.

D'un point de vue plus large, ce type de problème est représentatif de nombreuses situations en informatique, notamment dans les domaines de :

- l'intelligence artificielle (résolution de contraintes),
- la génération procédurale (création automatique de contenu),
- et les systèmes d'aide à la décision.

De plus, le contrôle de la difficulté d'un problème algorithmique est un enjeu important dans de nombreux domaines, allant du jeu vidéo à l'apprentissage adaptatif.

## 1.3 Approches possibles et choix retenu

Plusieurs approches peuvent être envisagées pour résoudre le problème de génération et de résolution de Sudoku :

### 1.3.1 Approches basées sur des techniques humaines

Ces méthodes reproduisent les stratégies utilisées par les joueurs humains (singletons, paires, etc.). Elles permettent de qualifier plus finement la difficulté d'une grille, mais sont souvent complexes à implémenter.

### 1.3.2 Approches algorithmiques classiques

Elles reposent sur des techniques comme :

- le backtracking,
- la recherche exhaustive avec contraintes,
- ou des heuristiques d'optimisation.

Ces méthodes sont robustes et garantissent la résolution complète des grilles.

### 1.3.3 Approches hybrides

Elles combinent les deux précédentes : un solveur automatique avec une analyse basée sur des techniques humaines pour évaluer la difficulté.

#### Choix retenu

Dans le cadre du Sprint 1, le choix s'est porté sur une approche principalement algorithmique, basée sur un solveur utilisant le backtracking.

Ce choix est justifié par :

- sa simplicité d'implémentation,
- sa fiabilité pour garantir une solution,
- et sa capacité à fournir des métriques de difficulté via le nombre de backtracks.

Comme précisé dans le rapport, la difficulté d'une grille est estimée à partir du nombre de retours arrière (backtracks) effectués lors de la résolution.

À terme, cette approche pourra être enrichie par des techniques de résolution humaines, prévues dans les sprints suivants.

## 1.4 Cahier des charges détaillé

Le cahier des charges du projet définit les fonctionnalités et contraintes principales du système.

### 1.4.1 Fonctionnalités principales

Le système doit permettre :

- la génération de grilles complètes valides,
- la création de grilles jouables avec solution unique,
- l'évaluation de la difficulté en fonction du processus de résolution,
- la résolution automatique des grilles,
- la lecture et l'écriture de grilles via un parseur de fichiers,
- ainsi qu'une interface console interactive permettant de jouer.

Le rapport précise que l'interface permet déjà :

- d'afficher une grille,
- de choisir une difficulté,
- d'insérer des valeurs,
- de voir les candidats possibles,
- et de vérifier leur validité.

### 1.4.2 Contraintes techniques

Le projet doit respecter plusieurs contraintes :

- développement en **Python**,
- utilisation de bibliothèques standards,
- exécution sur les environnements Linux de l'université,
- temps de calcul raisonnable pour la génération et la résolution,
- organisation du code via un dépôt Git structuré.

## 2 Technologies utilisées

### 2.1 Langages et outils utilisés

Dans le cadre de ce projet, plusieurs technologies et outils ont été mobilisés afin de concevoir, développer et organiser efficacement le système de génération et de résolution de grilles de Sudoku.

### 2.1.1 Langage de programmation

Le projet a été développé principalement en **Python**, qui constitue le cœur de l'implémentation. Ce langage est utilisé pour l'ensemble des composants du système :

- génération des grilles,
- résolution (solveur),
- évaluation de la difficulté,
- gestion des fichiers (parseur),
- interface utilisateur en console.

Comme indiqué dans le rapport de Sprint 1, Python a été choisi pour développer l'ensemble des modules du projet, en s'appuyant principalement sur des bibliothèques standards.

### 2.1.2 Bibliothèques utilisées

Le projet utilise essentiellement des bibliothèques natives de Python, notamment :

- **random** : pour la génération aléatoire des grilles,
- **os** : pour la gestion des fichiers et du système,
- **shutil** : pour certaines opérations sur les fichiers,
- **pickle** : permet de sauvegarder directement des objets Python dans un fichier,
- **matplotlib** : pour la visualisation des statistiques du solveur (comme illustré dans le graphique en page 5 du rapport).

L'utilisation de bibliothèques standards permet de garantir la simplicité, la portabilité et la reproductibilité du projet.

### 2.1.3 Outils de développement

**Gestion de version** Le projet est géré à l'aide de **Git**, avec un dépôt hébergé sur GitLab. Cela permet :

- de suivre l'évolution du code,
- de travailler en équipe de manière collaborative,
- de gérer différentes versions via des branches.

Le rapport précise l'adoption d'un workflow simple avec une branche principale et des branches de développement.

**Outils de communication** La communication au sein du groupe est assurée via :

- Discord, pour les échanges rapides,
- des réunions hebdomadaires, permettant de suivre l'avancement et répartir les tâches.

**Environnement d'exécution** Le projet est conçu pour être exécuté sur :

- les machines Linux de l'université,
- ainsi que les ordinateurs personnels des membres du groupe.

Cette contrainte impose une attention particulière à la portabilité du code.

## 2.2 Justification des choix technologiques

### 2.2.1 Choix du langage Python

Le choix de Python s'explique par plusieurs raisons majeures.

Tout d'abord, Python est un langage simple et lisible, ce qui facilite la compréhension du code, notamment dans un contexte de travail en groupe. Cette lisibilité est particulièrement importante pour un projet impliquant plusieurs modules (générateur, solveur, interface, parseur).

Ensuite, Python est particulièrement adapté à l'implémentation d'algorithmes complexes, comme le backtracking utilisé pour résoudre les grilles de Sudoku. Sa syntaxe permet de se concentrer sur la logique algorithmique sans être alourdi par des contraintes techniques.

De plus, Python offre une grande flexibilité dans la manipulation des structures de données, comme les matrices utilisées pour représenter les grilles. Cela facilite la mise en place rapide de prototypes fonctionnels, comme celui réalisé lors du Sprint 1.

Enfin, Python est largement utilisé dans le domaine académique, ce qui en fait un choix cohérent avec le cadre du projet.

## 3 Développements Logiciel : Conception, Modélisation, Implémentation

### 3.1 Présentation des développements réalisés

### 3.2 Modules principaux du logiciel

### 3.3 Diagramme de cas d'utilisation (UML)

### 3.4 Diagramme de classes (UML)

### 3.5 Fonctionnalités de l'interface (console)

Le projet propose une interface console interactive permettant à l'utilisateur d'interagir avec le système de génération et de résolution de Sudoku. Cette interface joue un rôle central dans le projet, car elle permet de tester les fonctionnalités développées tout en offrant une expérience utilisateur simple et fonctionnelle.

Au lancement du programme, un menu principal est affiché. L'utilisateur peut alors choisir entre générer une grille de Sudoku et jouer, ou quitter l'application. Ce point d'entrée est volontairement simple afin de rendre l'utilisation du programme intuitive.

Une fois le jeu lancé, l'utilisateur est invité à choisir un niveau de difficulté parmi plusieurs options : facile, moyen, difficile, extrême et un mode avancé (« God Mode »). Ce choix influence directement la génération de la grille, notamment le nombre de cases vides et la complexité de résolution.

Après la génération, la grille est affichée dans la console. L'interface fournit également des informations supplémentaires sur la difficulté, telles que :

- une estimation basée sur des méthodes de résolution humaines,
- le nombre d'étapes nécessaires pour résoudre la grille,
- la technique la plus complexe utilisée.

Ces informations permettent à l'utilisateur de mieux comprendre le niveau réel de la grille.

L'utilisateur peut ensuite interagir avec la grille via plusieurs actions. Il peut notamment entrer une valeur dans une case, en précisant la ligne, la colonne et la valeur souhaitée. Le système vérifie alors que la saisie est valide : les coordonnées doivent être correctes, la case ne doit pas être déjà remplie, et la valeur doit respecter les règles du Sudoku. Si une erreur est détectée, un message explicite est affiché.

Un système de gestion des erreurs est également mis en place. L'utilisateur dispose d'un maximum de trois erreurs ; au-delà de cette limite, la partie se termine automatiquement avec un message de type « Game Over ». Cela ajoute une dimension de contrainte et rend l'interaction plus dynamique.

L'interface propose également d'afficher les candidats possibles pour chaque case vide. Cette fonctionnalité permet de visualiser toutes les valeurs envisageables dans une case donnée, ce qui aide à comprendre le processus de résolution et peut servir d'outil pédagogique.

En complément, l'utilisateur peut choisir de résoudre automatiquement la grille selon deux approches différentes :

- une résolution complète par backtracking (méthode algorithmique),
- une résolution basée sur des techniques humaines, accompagnée de statistiques (nombre d'étapes, techniques utilisées, etc.).

Ces deux méthodes permettent de comparer les approches et d'analyser la difficulté de la grille générée.

Enfin, la partie se termine lorsque la grille est entièrement complétée correctement, lorsque l'utilisateur choisit de quitter, ou après une résolution automatique.

**Le chemin d'exécution est celui-ci :** Le chemin d'exécution a été modifié afin d'éviter la génération directe des grilles pendant l'utilisation normale du programme. Désormais, lorsqu'une partie est lancée, le programme ne génère plus une nouvelle grille en temps réel. Il charge une grille déjà préparée depuis la banque de grilles.

Le nouveau chemin d'exécution est le suivant :

```
InterfaceConsole.startPlaying() → playSudoku() → askDifficulty() →
BanqueGrilles.charger_grille_aleatoire(difficulty) → BanqueGrilles.charger_banque() →
pickle.load() → random.choice() → gameLoop()
```

La méthode `charger_banque()` lit le fichier `banque_grilles.pkl`, qui contient les grilles pré-générées. Ensuite, `charger_grille_aleatoire()` sélectionne au hasard une grille correspondant à la difficulté choisie par l'utilisateur. Cette grille contient déjà la grille de jeu, la solution complète et les statistiques de résolution calculées auparavant avec `SolverHumanStats`.

La génération complète existe encore, mais elle est maintenant utilisée uniquement avant la démonstration, dans le script de préparation :

```
PreparerBanque.py → GrilleBacktrack.generateEntireGrille() →
GrilleHuman.generateValuesHumanRated() → SolverHumanStats.solveWithStats() →
SolverHuman → BanqueGrilles.ajouter_grille() → BanqueGrilles.sauvegarder_banque() →
pickle.dump()
```

Cette étape permet de générer plusieurs grilles pour chaque difficulté, puis de les sauvegarder dans le fichier `banque_grilles.pkl`.

### 3.6 Structures de données principales

### 3.7 Format des entrées (grilles Sudoku)

### 3.8 Procédures de lecture et validation

### 3.9 Statistiques

Cette section présente quelques statistiques générales sur la taille du projet. Ces chiffres permettent d'avoir une vue d'ensemble de l'organisation du code et de la répartition des fichiers.

#### 3.9.1 Nombre de modules / classes

Le projet contient environ **35 fichiers Python**. Chaque fichier Python peut être considéré comme un module, car il regroupe une partie précise du programme : gestion des grilles, génération, résolution, interface, historique, difficultés, techniques de résolution ou encore sauvegarde des grilles pré-générées.

Les fichiers Python n'ont pas tous la même taille. Certains sont assez courts, avec environ une vingtaine de lignes, tandis que d'autres sont beaucoup plus importants et peuvent dépasser les 300 lignes. En moyenne, les fichiers Python contiennent environ **150 à 200 lignes**.

Le projet contient aussi plusieurs fichiers liés à l'interface web et aux données :

| Type de fichier | Nombre de fichiers | Taille approximative                                        |
|-----------------|--------------------|-------------------------------------------------------------|
| Python .py      | 35                 | 150 à 200 lignes en moyenne                                 |
| HTML            | 2                  | environ 30 lignes en moyenne                                |
| CSS             | 2                  | environ 70 lignes pour un fichier, environ 700 pour l'autre |
| JavaScript      | 1                  | environ 8 lignes                                            |
| JSON            | 2                  | environ 1050 lignes au total                                |

TABLE 1 – Répartition approximative des fichiers du projet

Cette organisation montre que le projet est principalement développé en Python, avec quelques fichiers supplémentaires pour l'interface web et la configuration.

#### 3.9.2 Nombre de lignes de code

En estimant le nombre de lignes à partir des fichiers du projet, on obtient environ :

- **Python** : 35 fichiers avec environ 150 à 200 lignes en moyenne, soit environ 5250 à 7000 lignes ;
- **HTML** : 2 fichiers avec environ 30 lignes chacun, soit environ 60 lignes ;
- **CSS** : 2 fichiers, avec environ 70 lignes pour l'un et 700 lignes pour l'autre, soit environ 770 lignes ;

- **JavaScript** : 1 fichier d'environ 8 lignes ;
  - **JSON** : 2 fichiers contenant environ 1050 lignes au total.
- On peut résumer cette estimation dans le tableau suivant :

| Type de fichier | Nombre approximatif de lignes |
|-----------------|-------------------------------|
| Python          | 5250 à 7000                   |
| HTML            | 60                            |
| CSS             | 770                           |
| JavaScript      | 8                             |
| JSON            | 1050                          |
| <b>Total</b>    | <b>7138 à 8888</b>            |

TABLE 2 – Estimation du nombre de lignes de code du projet

Le projet contient donc environ **7000 à 9000 lignes** au total. Cette estimation inclut le code, les commentaires, les lignes vides ainsi que les fichiers de données ou de configuration. Le nombre exact peut varier légèrement selon la méthode de comptage utilisée.

La majorité du code se trouve dans les fichiers Python, car ils contiennent toute la logique principale du projet : génération des grilles, résolution, classification par difficulté, gestion de l'historique, banque de grilles pré-générées et interaction avec l'utilisateur.

## 4 Algorithmes et Analyse

- Algorithmes principaux :
  - backtracking
  - génération de grille
- Illustration du fonctionnement
- Complexité temporelle :
  - backtracking
  - génération

## 5 Analyse des Résultats

- Graphiques de performance
- Analyse des résultats
- Comparaison des méthodes :
  - backtracking vs méthodes humaines
- Procédures de test :
  - génération de grilles
  - validation des solutions

## 6 Gestion du Projet

- Organisation du projet
- Diagramme de Gantt
- Répartition des tâches
- Changements en cours de projet

## 7 Bilan et Conclusions

- Fonctionnalités réalisées
- Comparaison avec cahier des charges
- Limites du projet

- Perspectives :
  - amélioration des algorithmes
  - interface graphique

## **8 Bibliographie**

- Ressources utilisées
- Articles scientifiques
- Documentation

## **9 Annexes**

- Fragments de code
- Manuel d'utilisation
- Exemples de grilles